



Instituto Tecnológico Centroamericano

Docente: Ing. Alfredo Alcides Aquino Aguilar

Proyecto: Sistema de control de accesos biométrico con reconocimiento facial

Estudiantes: Melgares Valdez Henry Levi.

Hernández Ramírez Ronald Alexis.

índice

Desarrollo Backend + Base de Datos	3
4.1 Objetivo de la etapa	3
4.2 Alcance técnico del backend	3
4.3 Tecnologías a utilizar	3
4.4 Arquitectura del Backend (módulos)	4
4.5 Registro y Enrolamiento Previo del Usuario (Requisito Obligatorio)	5
4.5.1 Registro administrativo	5
4.5.2 Enrolamiento biométrico facial	6
4.5.3 Habilitación del método alternativo (PWA – OTP)	6
4.6 Flujo principal del acceso (Reconocimiento Facial)	7
4.7 Flujo alternativo (PWA con Código Dinámico)	8
4.8 Diseño de Base de Datos (tablas, campos y relaciones)	8
4.8.1 Objetivo del diseño	8
4.8.2 Tablas principales	9
4.8.3 Relaciones resumidas (para explicar en el documento)	13
4.9 Endpoints principales de la API REST	14
4.9.1 Autenticación (JWT)	14
4.9.2 Administración (Usuarios / Permisos / Catálogos)	14
4.9.3 Enrolamiento biométrico (Azure AI Face)	15
4.9.4 Acceso principal (Reconocimiento facial)	15
4.9.5 Acceso alternativo (PWA – OTP)	15
4.9.6 Registros y reportes (Dashboard)	16
4.10 Manejo y protección de datos	17
4.11 Resultados esperados de la etapa	17
Conclusión de la Etapa 4	17

Desarrollo Backend + Base de Datos

4.1 Objetivo de la etapa

Implementar la lógica central del sistema de control de accesos biométrico facial mediante el desarrollo de un backend en Python, la creación de una API REST, la integración con Azure AI Face para verificación biométrica, la conexión con una base de datos relacional y el manejo seguro de datos administrativos y eventos de acceso. Además, se incorpora un método alternativo de autenticación basado en una PWA que genera códigos dinámicos temporales para permitir el ingreso cuando el reconocimiento facial no esté disponible, manteniendo como requisito que el usuario esté registrado previamente.

4.2 Alcance técnico del backend

El backend será responsable de:

- Gestión de usuarios (registro, actualización, estado activo/inactivo).
 - Gestión biométrica (asociar usuario con identificador biométrico de Azure AI Face).
 - Gestión de aulas y horarios (definición de permisos por aula/tiempo).
 - Validación de acceso en tiempo real:
 - Por reconocimiento facial (principal).
 - Por código dinámico PWA (alternativo).
 - Registro de eventos (permitido/denegado + motivo + hora + aula).
 - Auditoría y seguridad (JWT, roles, logs, cifrado de datos sensibles).
 - Reportería básica (consultas y estadísticas para panel web).
-

4.3 Tecnologías a utilizar

- Lenguaje: Python
 - Framework API: DJANGO DRF
 - Base de datos: Azure PostgreSQL, Azure Storage
 - Autenticación: JWT + control de roles (RBAC)
 - Servicio biométrico: Azure AI Face (Cloud)
 - Método alternativo: PWA + Código Dinámico (OTP/TOTP)
-

4.4 Arquitectura del Backend (módulos)

El backend se organiza en módulos para facilitar el mantenimiento y escalabilidad:

1. Módulo de Autenticación
 - o Login del administrador
 - o Tokens JWT
 - o Roles y permisos
 2. Módulo de Usuarios
 - o CRUD de usuarios
 - o Estados (activo/inactivo)
 - o Asociación biométrica
 3. Módulo de Biometría Facial (Azure AI Face)
 - o Envío de imagen para verificación
 - o Recepción de coincidencia y score
 4. Módulo de Permisos y Horarios
 - o Validar si el usuario tiene permiso para esa aula
 - o Validar si está dentro del horario
 5. Módulo de Accesos
 - o Permitir/denegar
 - o Registrar eventos
 - o Motivos de denegación
 6. Módulo PWA – Código Dinámico
 - o Generación de código temporal (OTP)
 - o Validación del código ingresado
 - o Control de expiración y uso único
 7. Módulo de Reportes
 - o Listado de accesos
 - o Estadísticas básicas por día/usuario/aula
-

4.5 Registro y Enrolamiento Previo del Usuario (Requisito Obligatorio)

Antes de que un usuario pueda ingresar mediante reconocimiento facial o código dinámico OTP, debe realizarse un proceso previo de registro y enrolamiento en el sistema.

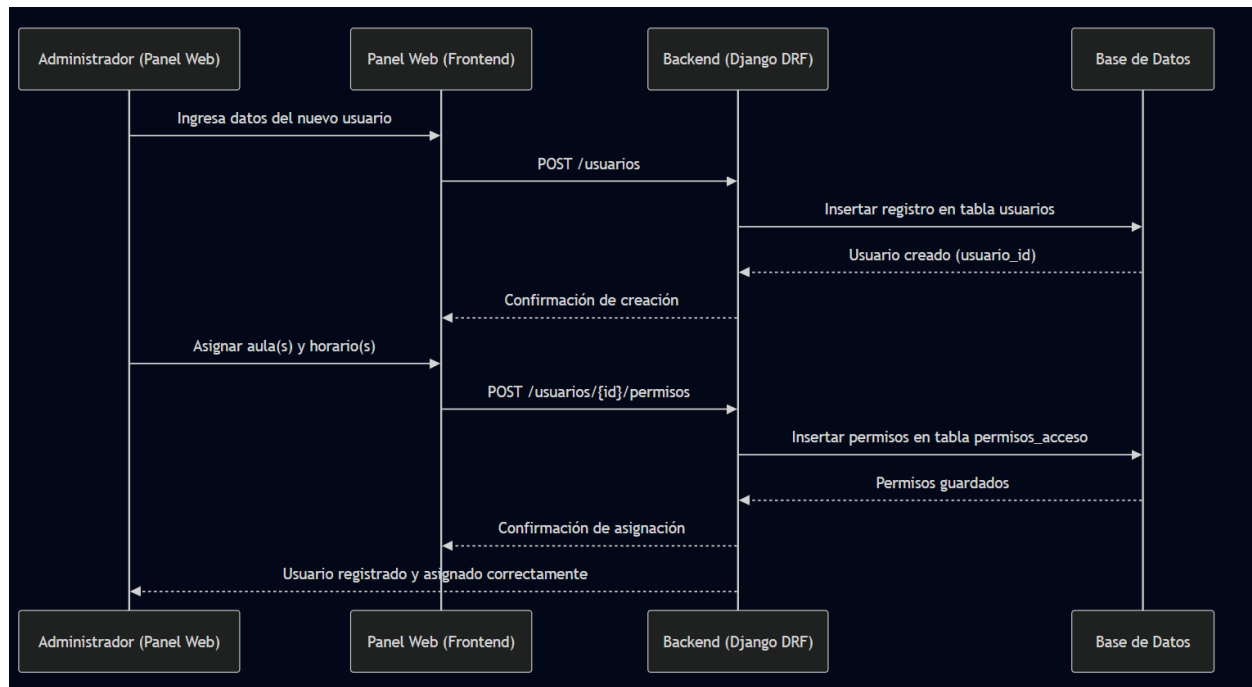
Este proceso garantiza que únicamente personas autorizadas puedan autenticarse por cualquiera de los métodos disponibles.

4.5.1 Registro administrativo

El administrador del sistema realiza:

1. Creación del usuario en el sistema.
2. Asignación de rol (docente, seguridad, administrado, Biometrico).
3. Definición de estado (activo/inactivo).
4. Asignación de aula(s) permitida(s).
5. Asignación de horario(s) de acceso.

El usuario queda registrado en la tabla **usuarios** y vinculado a sus permisos correspondientes.

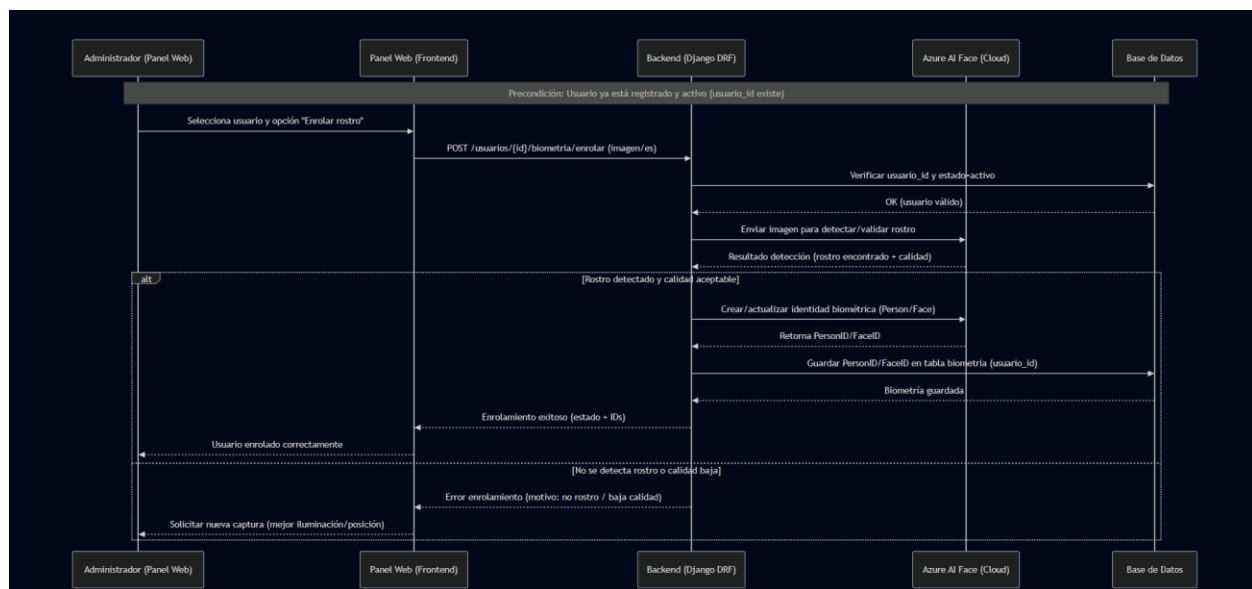


4.5.2 Enrolamiento biométrico facial

Una vez registrado el usuario:

1. Se capturan imágenes del rostro.
2. El backend envía la información a Azure AI Face.
3. Azure genera un identificador biométrico (Person ID / Face ID).
4. El sistema almacena únicamente la referencia biométrica en la tabla **biometria**.

Este proceso permite que posteriormente el sistema pueda reconocer al usuario durante la validación facial.



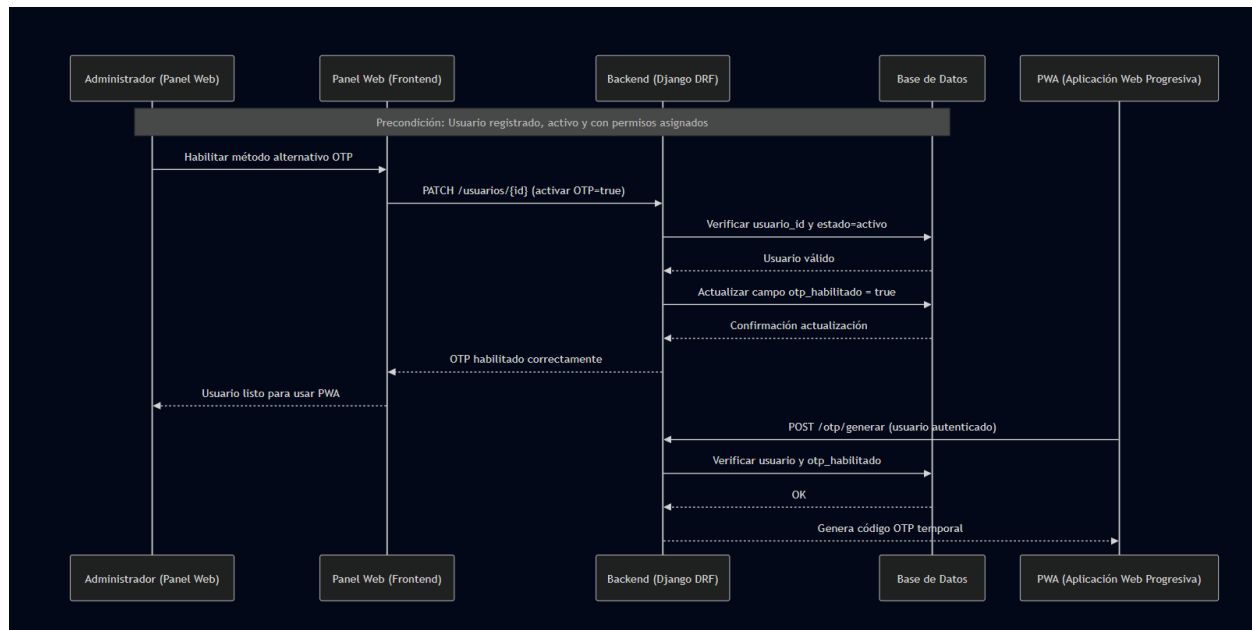
4.5.3 Habilitación del método alternativo (PWA – OTP)

El acceso mediante código dinámico también requiere registro previo:

- El usuario debe existir y estar activo.
- Debe tener permisos asignados.
- Debe estar dentro del horario autorizado.

La PWA genera un código temporal asociado al usuario registrado, el cual será validado por el backend antes de autorizar el acceso.

Sin este proceso previo, ningún método de autenticación será válido.



4.6 Flujo principal del acceso (Reconocimiento Facial)

1. La cámara captura la imagen del rostro.
2. El backend recibe la imagen o referencia.
3. El backend envía la imagen a Azure AI Face.
4. Azure retorna resultado (usuario + score o no match).
5. El backend consulta la BD:
 - o ¿Usuario existe y está activo?
 - o ¿Tiene permiso para el aula?
 - o ¿Está en horario válido?
6. Se responde:
 - o Acceso permitido
 - o Acceso denegado (se indica motivo)
7. Se registra el evento en la BD.

4.7 Flujo alternativo (PWA con Código Dinámico)

Este método se utiliza cuando:

- falla la cámara
- iluminación deficiente
- error temporal del servicio biométrico
- o el usuario no fue reconocido correctamente

Flujo:

1. El usuario abre la PWA (requiere estar registrado).
 2. Solicita un código dinámico temporal.
 3. Ingresa el código en el punto de acceso o interfaz.
 4. Backend valida:
 - o Código pertenece al usuario
 - o Código no expiró
 - o Código no fue usado
 - o Usuario tiene permiso y horario válido
 5. Si es válido → Acceso permitido y se registra evento.
-

4.8 Diseño de Base de Datos (tablas, campos y relaciones)

4.8.1 Objetivo del diseño

La base de datos relacional (PostgreSQL) almacena: usuarios/roles, credenciales de PWA, enrolamiento facial (Azure Face ID), permisos por aula y horario, y el historial completo de accesos con su resultado y motivo. Esto permite trazabilidad, auditoría y reportes (como se plantea en el flujo de datos) .

4.8.2 Tablas principales

1) rol

Propósito: Catálogo de roles del sistema (Admin, Subadmin, Docente, etc.).

Campos recomendados:

- id (PK)
- nombre_rol (único)

Relación: 1 rol ↔ muchos usuarios (vía tabla puente usuario_rol).

2) usuario

Propósito: Identidad administrativa del usuario (docente, seguridad, admin).

Campos recomendados:

- id (PK)
- nombre
- apellido
- dui (opcional/único si aplica)
- email (único)
- telefono_alternativo (opcional)
- residencia (opcional)
- activo (boolean)
- created_at

Relación:

- usuario ↔ roles (muchos a muchos vía usuario_rol)
 - usuario ↔ biometría (1 a 1 o 1 a N, según re-enrolamiento)
 - usuario ↔ accesos (1 a N)
 - usuario ↔ credenciales/sesiones (PWA)
-

3) usuario_rol (tabla puente)

Propósito: Asignación de roles a usuarios (RBAC).

Campos recomendados:

- id (PK)
 - usuario_id (FK → usuario.id)
 - rol_id (FK → rol.id)
-

4) credencial (PWA)

Propósito: Credenciales de autenticación para el usuario en la PWA (según tu diagrama aparece como soporte para PWA) .

Campos recomendados:

- id (PK)
 - usuario_id (FK → usuario.id)
 - username (único)
 - password_hash (nunca guardar password en texto plano)
 - password_updated_at
-

5) sesion

Propósito: Control de sesiones activas (útil para seguridad, auditoría y expiración).

Campos recomendados:

- id (PK)
 - usuario_id (FK → usuario.id)
 - token_jwt_id (o refresh_token_id si lo manejas así)
 - fecha_inicio
 - fecha_fin (opcional)
 - activo (boolean)
-

6) aula

Propósito: Catálogo de aulas/puntos de acceso.

Campos recomendados:

- id (PK)
 - codigo (único)
 - descripcion
 - activo (boolean)
-

7) horario

Propósito: Ventanas de tiempo autorizadas (día y rango de horas).

Campos recomendados:

- id (PK)
 - dia_semana (0–6 o texto “Lunes...Domingo”)
 - hora_inicio
 - hora_fin
-

8) permiso_acceso

Propósito: Permisos por usuario + aula + horario (control institucional).

Campos recomendados:

- id (PK)
- usuario_id (FK → usuario.id)
- aula_id (FK → aula.id)
- horario_id (FK → horario.id)
- activo (boolean)

Índices recomendados:

- índice compuesto: (usuario_id, aula_id, horario_id)
 - índice por usuario_id para consultas rápidas
-

9) biometria

Propósito: Referencia biométrica del enrolamiento facial (Azure AI Face ID/PersonID). En tu diagrama se menciona explícitamente “ID devuelto por Azure AI Face” .

Campos recomendados:

- id (PK)
- usuario_id (FK → usuario.id)
- face_id (Azure Face ID / Person ID)
- storage_url (opcional si guardan evidencia en Azure Storage)
- timestamp (fecha de enrolamiento)
- estado (activo/revocado)

Buenas prácticas:

- Guardar **solo la referencia** (FaceID/PersonID), no imágenes sin cifrado.
-

10) acceso_evento

Propósito: Historial de accesos (auditoría). En tu arquitectura de analítica se toma esta tabla como fuente de datos para KPIs .

Campos recomendados:

- id (PK)
- usuario_id (FK → usuario.id)
- aula_id (FK → aula.id)
- fecha_hora (timestamp)
- metodo (Facial / OTP(PWA) / Huella / Manual)
- resultado (Éxito / Fallo)
- motivo (Fuera de horario, Sin permiso, No match, OTP expirado, etc.)
- score (confianza facial, si aplica)
- alerta (boolean)
- created_at

Índices recomendados:

- (fecha_hora)
 - (usuario_id, fecha_hora)
 - (aula_id, fecha_hora)
-

11) codigos_dinamicos (equivalente a PIN_CONTINGENCIA)

Propósito: Respaldo cuando falla el reconocimiento facial (PWA genera código temporal). En tu diagrama aparece como módulo/tabla de contingencia (PIN) , aquí lo formalizamos como OTP.

Campos recomendados:

- id (PK)
 - usuario_id (FK → usuario.id)
 - codigo_hash (guardar hash, no el código en claro si es posible)
 - expira_en (timestamp)
 - usado (boolean)
 - usado_en (timestamp, opcional)
 - intentos (opcional, para bloqueo)
 - created_at
-

4.8.3 Relaciones resumidas (para explicar en el documento)

- usuario ↔ rol (M:N mediante usuario_rol)
- usuario → biometria (1:1 o 1:N)
- usuario → permiso_acceso (1:N)
- permiso_acceso → aula (N:1) y → horario (N:1)
- usuario → acceso_evento (1:N) y aula → acceso_evento (1:N)
- usuario → codigos_dinamicos (1:N)

Todo esto encaja con la arquitectura en capas y servicios, donde el backend consulta permisos/horarios, valida biometría, registra evento y luego se visualiza en dashboards

4.9 Endpoints principales de la API REST

Basado en la **arquitectura basada en servicios** (Auth Service, User Service CRUD, PIN/OTP Service, Access Decision Service, Event Log, Analytics) , el esquema recomendado queda así:

4.9.1 Autenticación (JWT)

Endpoint	Método	Actor	Entrada	Salida	Seguridad
/auth/login	POST	Admin / Usuario PWA	username + password	access + refresh	Público
/auth/refresh	POST	Admin / Usuario PWA	refresh_token	access nuevo	Público
/auth/me	GET	Admin / Usuario PWA	—	perfil + roles	JWT

4.9.2 Administración (Usuarios / Permisos / Catálogos)

Endpoint	Método	Actor	Entrada	Salida	Seguridad
/usuarios	POST	Admin	datos usuario	usuario_id	JWT + rol Admin
/usuarios/{id}	PUT/PATCH	Admin	cambios	usuario actualizado	JWT + Admin
/usuarios/{id}/permisos	POST	Admin	aula_id + horario_id	permiso creado	JWT + Admin
/aulas	POST	Admin	codigo + descripción	aula creada	JWT + Admin
/horarios	POST	Admin	día + rango	horario creado	JWT + Admin

(Opcional para panel admin)

- /usuarios GET, /aulas GET, /horarios GET, /usuarios/{id} GET

4.9.3 Enrolamiento biométrico (Azure AI Face)

Endpoint	Método	Actor	Entrada	Salida	Seguridad
/usuarios/{id}/biometria/enrolar	POST	Admin	imagen(es) rostro	face_id/person_id	JWT + Admin

Este endpoint alimenta la tabla biometria con el FaceID devuelto por Azure AI Face .

4.9.4 Acceso principal (Reconocimiento facial)

Endpoint	Método	Actor	Entrada	Salida	Seguridad
/accesos/validar-facial	POST	Terminal/Punto de acceso	imagen + aula_id	permitido/denegado + motivo + score	JWT (device) o API Key

Reglas internas (importantes para el documento):

- Verifica usuario activo
- Verifica permiso y horario
- Registra en acceso_evento con método = “Facial”

4.9.5 Acceso alternativo (PWA – OTP)

Endpoint	Método	Actor	Entrada	Salida	Seguridad
/otp/generar	POST	Usuario (PWA)	sesión JWT	otp + expiración	JWT
/otp/validar	POST	Terminal/Punto de acceso	otp + aula_id	permitido/denegado + motivo	JWT (device) o API Key

Reglas internas:

- OTP no expirado, no usado
- Usuario activo + permiso + horario

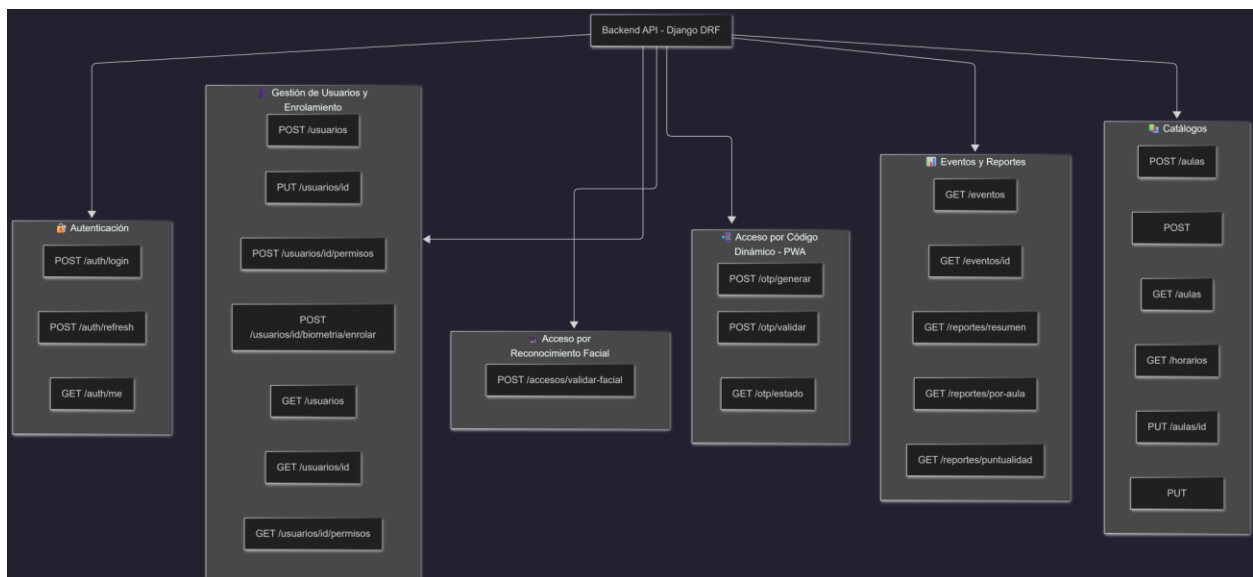
- Registra evento en acceso_evento con método = “OTP(PWA)”

Esto corresponde al “PIN Service – Contingencia” en tu arquitectura por servicios .

4.9.6 Registros y reportes (Dashboard)

Endpoint	Método	Actor	Salida	Seguridad
/eventos	GET	Admin / Seguridad	listado eventos (filtros)	JWT + rol
/reportes/resumen	GET	Admin / Subadmin	KPIs/estadísticas	JWT + rol

En tu arquitectura de analítica, los KPIs salen desde datos estructurados con base en eventos y se visualizan en dashboard/alertas .



4.10 Manejo y protección de datos

- No almacenar fotos sin cifrado (solo si es necesario y con permisos).
 - Guardar identificadores biométricos y registros de eventos.
 - Uso de JWT y roles (admin/seguridad/autoridad).
 - Auditoría de acciones críticas.
 - Validaciones de entrada para prevenir inyección y ataques comunes.
 - Expiración y uso único de códigos dinámicos.
-

4.11 Resultados esperados de la etapa

- Backend en Python funcional.
 - API REST documentada y segura.
 - Base de datos implementada con integridad referencial.
 - Integración con Azure AI Face.
 - Registro completo de accesos e intentos fallidos.
 - Método alternativo PWA con código dinámico operativo.
 - Base lista para integrar el panel web y el despliegue del aula piloto.
-

Conclusión de la Etapa 4

El desarrollo del backend y la base de datos constituye el núcleo del sistema, permitiendo la autenticación biométrica facial, el control por permisos y horarios, y el almacenamiento seguro de información. Con la incorporación del método alternativo mediante PWA con códigos dinámicos temporales y el proceso previo de registro y enrolamiento obligatorio, se garantiza que solo usuarios previamente autorizados puedan acceder al sistema, fortaleciendo la seguridad y la confiabilidad de la solución implementada.